

CSE305 project: Map-reduce on recursively enumerated sets

Ricardo Buring ricardo.buring@polytechnique.edu
Augustin Albert augustin.albert@polytechnique.edu

For sets of objects defined in a recursive manner, there are efficient ways to distribute computations such as counting the total number of elements (or elements with a specific property), finding an element satisfying a certain property, or checking that all elements satisfy some property (or that none do). This has applications to combinatorics, and to constraint satisfaction problems, for example solving Sudoku grids.

- *Recursively enumerated sets.* Let U be a set and let **seeds** be a subset of U . Let **successors**: $U \rightarrow 2^U$ be a function associating to each element of U a subset of U . Let S be the set of all elements of U that can be reached from a seed in **seeds** by applying recursively the **successors** function; the set S is said to be *recursively enumerated*.
- *Map-reduce.* The type of computations we are interested in can be expressed as applying a function **map_function**: $U \rightarrow A$ to every element of a recursively enumerated set S and combining the results together using a function **reduce_function**: $A \times A \rightarrow A$. For example:
 - **map_function**(x) = 1 and **reduce_function**(x, y) = $x + y$ to compute $|S|$,
 - **map_function**(x) = 1 if x satisfies property $P(x)$ and 0 otherwise;
reduce_function(x, y) = $x + y$ to compute $|\{x \in S \mid x \text{ satisfies property } P(x)\}|$.
 - **map_function**(x) = x and **reduce_function**(x, y) = $\max(x, y)$ to find the maximal element of S under some ordering.
 - **map_function**(x) = **true** if x satisfies property $Q(x)$ and **false** otherwise;
reduce_function(x, y) = $x \wedge y$ to check whether all elements of S satisfy property Q .

The reduction operation should be associative to allow parallel evaluation, and the recursion over the set S can be performed depth-first to minimize memory usage.

- *Work-stealing.* Work-stealing is a scheduling strategy where idle worker threads dynamically “steal” tasks from the queues of busy workers, enabling efficient load balancing in irregular parallel computations [2]. When exploring a recursively enumerated set, the search tree may be highly unbalanced; work-stealing allows idle threads to take unexplored branches from others.
- *Implementation.* The goal of this project is to implement a generic parallel map-reduce algorithm on recursively enumerated sets by using work-stealing. You can do this in C++ with threads by creating a `RecursivelyEnumeratedSet` class that takes **seeds** and **successors** as input in its constructor, and implementing a **map_reduce** member function that takes **map_function** and **reduce_function** as input and returns the result of the map-reduce operation applied to the recursively enumerated set. That is, you can mimic the existing interface in the SAGEMATH computer algebra system [1]. See the section *How does it work?* of [1] for a description of their implementation; note however that it uses multiple processes and not just threads. You should ensure that your implementation passes tests including correctness on small sets, thread safety under concurrent access, and performance scaling measurements on larger enumeration problems.
- *Applications.* To demonstrate the effectiveness of your implementation, you can apply it to concrete problems, for example:

- *Sudoku solving*: Model the set of partial Sudoku grids where `seeds` contains the initial puzzle and `successors` fills one additional empty cell with a valid digit. Use map-reduce to count the number of valid completions, find a solution, or verify uniqueness.
- *Graph exploration*: Traverse all paths in a graph from a source node without revisiting vertices. Count Hamiltonian paths or find the longest simple path between two nodes.

References

- [1] SageMath documentation. Parallel computations using `RecursivelyEnumeratedSet` and `Map-Reduce`. https://doc.sagemath.org/html/en/reference/parallel/sage/parallel/map_reduce.html.
- [2] Wikipedia contributors. Work stealing. https://en.wikipedia.org/wiki/Work_stealing. Accessed: 2026-04-20.